

BACCALAURÉAT GÉNÉRAL

Sujet 04- Session 2026

NUMERIQUE ET SCIENCES INFORMATIQUES

ÉPREUVE DU SECOND GROUPE

DURÉE DE L'ÉPREUVE : **20 minutes**

DURÉE DE PRÉPARATION : **20 minutes**

L'utilisation d'une calculatrice est interdit

Le candidat préparera les différentes questions posées dans le sujet. Ces questions portent sur tout ou partie du programme du cycle terminale de la spécialité NSI.

Le candidat pourra répondre directement sur le sujet ou préparer les réponses aux questions sur une feuille de brouillon.

A la fin de la préparation, le candidat se présentera devant le jury muni de ce sujet ainsi que de toutes ses feuilles de brouillon.

Exercice 1 ►

Écrire une fonction `ajoute_dictionnaires` qui prend en paramètres deux dictionnaires `d1` et `d2` dont les clés sont des nombres et renvoie le dictionnaire `d` défini de la façon suivante :

- les clés de `d` sont celles de `d1` et celles de `d2` réunies;
- si une clé est présente dans les deux dictionnaires `d1` et `d2`, sa valeur associée dans le dictionnaire `d` est la somme de ses valeurs dans les dictionnaires `d1` et `d2`;
- si une clé n'est présente que dans un des deux dictionnaires, sa valeur associée dans le dictionnaire `d` est la même que sa valeur dans le dictionnaire où elle est présente.

Exemples :

```
1 >>> ajoute_dictionnaires({1: 5, 2: 7}, {2: 9, 3: 11})
2 {1: 5, 2: 16, 3: 11}
3 >>> ajoute_dictionnaires({}, {2: 9, 3: 11})
4 {2: 9, 3: 11}
5 >>> ajoute_dictionnaires({1: 5, 2: 7}, {})
6 {1: 5, 2: 7}
```

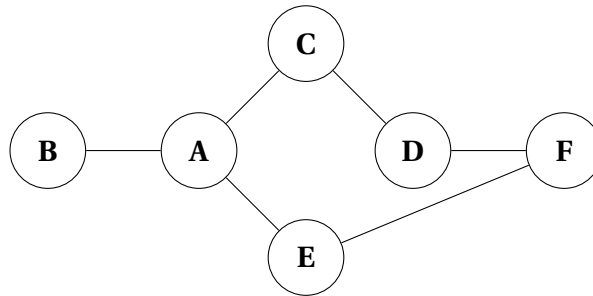
</> Code Python

```
def ajoute_dictionnaires(d1, d2):
    ''' Renvoie un dictionnaire d dont les clés sont celles de d1
    et celles de d2 réunies, et dont les valeurs sont la somme des
    valeurs associées à chaque clé dans d1 et d2.
    Si une clé n'est présente que dans un des deux dictionnaires,
    sa valeur associée dans le dictionnaire d est la même que
    sa valeur dans le dictionnaire où elle est présente. '''
    d = {}

    assert ajoute_dictionnaires({1: 5, 2: 7}, {2: 9, 3: 11}) == {1: 5, 2: 16, 3: 11}
    assert ajoute_dictionnaires({}, {2: 9, 3: 11}) == {2: 9, 3: 11}
    assert ajoute_dictionnaires({1: 5, 2: 7}, {}) == {1: 5, 2: 7}
```

Exercice 2 ►

Le graphe suivant modélise un ensemble de routeurs; les sommets sont les routeurs, les arêtes les liaisons entre eux-ci.



On désire parcourir ce graphe en largeur depuis le sommet A. On considère que les sommets sont explorés dans l'ordre alphabétique.

- Donner l'ordre de parcours des sommets du graphe avec un parcours en largeur depuis le sommet A.

- On rappelle qu'en protocole RIP, le nombre de saut entre un routeur et un autre est le nombre de liaisons minimales entre ces deux routeurs. Compléter la table de routage de A :

Table de routage de A		
routeur	nombre de sauts	prochain routeur
B	1	
C		
D		
E		
F		

On modélise les différents routeurs par des instances de la classe Routeur. Chaque routeur possède un attribut voisins qui est une liste contenant les routeurs directement connectés à lui. On suppose que les routeurs sont reliés entre eux conformément au graphe de la figure.

Code Python

```

1 class Routeur:
2     def __init__(self, nom):
3         self.nom = nom
4         self.voisins = []
5     def relie(self, autre_routeur):
6         # A compléter question 3
7
8
9
10
11
12
13 #
  
```

- nom : une chaîne de caractères qui identifie le routeur.
- voisins : une liste d'objets de type Routeur. Il s'agit de routeurs qui sont directement connectés au routeur.

3. Compléter le code de la méthode `relie` de la classe `Routeur` afin de permettre aux routeurs d'être reliés entre eux.
4. Ecrire les instructions permettant de créer les routeurs A, B, C, D, E et F. On supposera que le nom du routeur est le même que la lettre qui l'identifie. Ainsi le routeur A aura pour nom la chaîne de caractères 'A'.

</> Code Python

A compléter pour les routeurs A, B, C, D, E et F



#

5. Ecrire les instructions permettant de relier les routeurs entre eux conformément au graphe de la figure. On pourra utiliser la méthode `relie` de la classe `Routeur`. Si la liaison existe déjà, la méthode `relie` ne fait rien.

</> Code Python

A compléter pour créer le réseau de la figure 1



#

Exercice 3 ►

Compléter la fonction `fibonacci(n)` qui calcule le n -ième terme de la suite de Fibonacci de manière récursive. On rappelle que la suite de Fibonacci est définie par les relations suivantes :

$$fibonacci(0) = 0, \quad fibonacci(1) = 1, \quad fibonacci(n) = fibonacci(n-1) + fibonacci(n-2) \text{ pour } n \geq 2.$$

</> Code Python

```
def fibonacci(n):
    ''' Renvoie le n-ième terme de la suite de Fibonacci. '''
```



```
assert fibonacci(0) == 0
assert fibonacci(1) == 1
assert fibonacci(7) == 13
```